Daily Problem: March 7, 2026

Problem Statement

A directed graph with n vertices $\{0, \ldots, n-1\}$ can be represented by a word over $\{0, 1\}$ of length n^2, whose kth letter is 1 if and only if there is an edge from vertex i to vertex j, where

$$k = n \cdot i + j + 1.$$

Construct a Turing machine that recognizes the language of all those words over $\{0, 1\}$ that represent a graph with a path from vertex 0 to vertex $n-1$.

Solution

We construct a Turing machine that, given a binary string encoding a directed graph, accepts if and only if there is a path from vertex 0 to vertex $n-1$.

The main idea is the standard graph-reachability procedure: starting from vertex 0, repeatedly mark all vertices reachable by one more edge from already marked vertices. If vertex $n-1$ is eventually marked, accept; otherwise reject.

Since the graph is encoded as an $n \times n$ adjacency matrix written row by row, the Turing machine must first verify that the input length is a perfect square, so that the input indeed corresponds to some graph on n vertices.

Step 1: Verify that the input has length n^2 for some n

Let the input be a word

$$w \in \{0, 1\}^*.$$

The machine first checks whether $|w|$ is a perfect square.

If not, then w does not encode any graph of the required form, so the machine rejects.

If yes, then there exists a unique integer n such that

$$|w| = n^2.$$

The machine stores or computes this value n on a work portion of the tape.

Thus the input may now be viewed as an adjacency matrix

$$A = (a_{ij})_{0 \le i,j \le n-1},$$

where

$$a_{ij} = 1 \iff \text{there is an edge from } i \text{ to } j.$$

Step 2: Set up a reachable/unreachable marking for vertices

The machine creates a work area containing one marker for each vertex:

$$0, 1, 2, \ldots, n-1.$$

Next to each vertex it keeps a mark indicating whether that vertex is currently known to be reachable from vertex 0.

Initially, only vertex 0 is marked reachable.

So the initial reachable set is

$$R_0 = \{0\}.$$

All other vertices are marked unvisited or not yet known to be reachable.

Step 3: Repeatedly expand the set of reachable vertices

The machine now performs the following loop.

For each vertex i currently marked reachable, scan across row i of the adjacency matrix. That row consists of the n entries

$$a_{i0}, a_{i1}, \ldots, a_{i,n-1}.$$

Whenever the machine sees that

$$a_{ij} = 1,$$

it marks vertex j as reachable.

In other words, if i is already known to be reachable and there is an edge $i \to j$, then j is also reachable.

After scanning all reachable vertices and marking all newly discovered ones, the machine compares the new set of reachable vertices with the old one.

- If some new vertex was marked in this round, the machine repeats the process.
- If no new vertex was marked, then the process has stabilized.

This is exactly the usual breadth-first-search or fixed-point reachability computation, implemented on a Turing machine.

Step 4: Accept if and only if vertex $n-1$ becomes reachable

During or after each round, the machine checks whether vertex $n-1$ has been marked reachable.

- If vertex $n-1$ is marked reachable, the machine accepts.
- If the procedure stabilizes and vertex $n-1$ is still unmarked, the machine rejects.

Step 5: Why this construction is implementable on a Turing machine

We now explain why every part of the above procedure can be carried out by a Turing machine.

(a) Computing the rows of the matrix. Since the input is written row by row, row i consists of the block of symbols whose positions run from

$$ni + 1 \quad \text{to} \quad ni + n.$$

A Turing machine can find the beginning of row i by counting in binary or unary on a work tape, then scan across the next n symbols.

(b) Storing reachable vertices. A Turing machine can use an auxiliary region of the tape to keep a list of vertices together with a mark such as:

$$(i, \text{reachable}) \quad \text{or} \quad (i, \text{unreachable}).$$

(c) Repeating until no new vertex appears. The machine can keep a special flag indicating whether some new vertex was marked during the current pass. If the flag remains unset after a full pass, the machine halts and rejects unless vertex $n-1$ has already been found.

Hence the whole construction is effective and can be implemented by an ordinary single-tape or multitape Turing machine.

Step 6: Correctness

We now prove that the machine accepts exactly those encodings of graphs having a path from vertex 0 to vertex $n-1$.

Claim. After the tth round of the marking procedure, the set of marked vertices is exactly the set of vertices reachable from 0 by a path of length at most t.

Proof. We argue by induction on t.

For $t = 0$, only vertex 0 is marked. This is exactly the set of vertices reachable from 0 by a path of length 0.

Now assume the statement holds after round t. In round $t+1$, the machine considers every marked vertex i, that is, every vertex reachable by a path of length at most t, and marks every vertex j such that there is an edge $i \to j$. Therefore every newly marked vertex is reachable by a path of length at most $t+1$.

Conversely, if a vertex j is reachable from 0 by a path of length at most $t+1$, then either it is already reachable by a path of length at most t, or there exists a vertex i reachable by a path of length at most t such that $i \to j$ is an edge. By the induction hypothesis, i is marked by round t, and so the machine marks j in round $t+1$.

Thus the claim holds for all t. □

Since a graph with n vertices has at most n reachable-vertex discoveries, the process must stabilize after finitely many rounds. At that point, the marked vertices are exactly all vertices reachable from 0.

Therefore:

- if there is a path from 0 to $n-1$, then vertex $n-1$ will eventually be marked and the machine accepts;
- if there is no such path, then vertex $n-1$ will never be marked, and when the process stabilizes the machine rejects.

Conclusion

We have constructed a Turing machine that first checks whether the input length is of the form n^2, interprets the input as the adjacency matrix of a directed graph on vertices $\{0, \ldots, n-1\}$, and then computes the set of vertices reachable from vertex 0 by repeatedly following outgoing edges. It accepts exactly when vertex $n-1$ is reachable.

Hence the language of all binary strings encoding directed graphs with a path from vertex 0 to vertex $n-1$ is Turing-recognizable, and indeed decidable.